

Zintegrowany system autonomicznego nawadniania i monitorowania roślin domowych (WATIR IoT)

Wojciech Stochmiałek, Bartłomiej Raj, Marcel Podlecki, Bartłomiej Jedyk, Dominik Pokorski,
dr inż. Jacek Nazdrowicz – opiekun projektu

Politechnika Łódzka, WEEIA, Informatyka, Łódź, Polska

STRESZCZENIE Raport opisuje projekt WATIR IoT - hybrydowy system IoT do autonomicznego nawadniania i monitorowania roślin domowych. System opiera się na jednostce Arduino UNO R4 WiFi z czujnikami wilgotności gleby, klimatu (BME280) i poziomu wody (HC-SR04) oraz ruchomym ramieniem Pan-Tilt, pozwalającym z jednego stanowiska precyzyjnie podlewać wiele doniczek. Warstwę chmurową stanowi konteneryzowany backend Node.js/PostgreSQL, a warstwę użytkową natywna aplikacja Android (Kotlin, MVVM). Zastosowano hybrydowy model komunikacji: sterowanie czasu rzeczywistego odbywa się lokalnie, telemetria i konfiguracja profili - przez chmurę. W toku prac zmigrowano architekturę z układu dwupłytkowego (Arduino Uno + ESP8266) na jednopłytkowy Arduino R4 WiFi, rozwiązując problemy zasilania, zakłóceń elektromagnetycznych i konfliktów magistrali. Prototyp potwierdza wykonalność ekonomicznego modelu „jedna stacja – wiele roślin” wobec kul nawadniających i dedykowanych smart doniczek.

KEYWORDS Arduino UNO R4 WiFi, automatyzacja nawadniania, BME280, Docker, Internet Rzeczy (IoT), Kotlin, PostgreSQL, serwomechanizm Pan-Tilt, telemetria środowiskowa

I. WSTĘP

A. Obszar / tło działań

Utrzymanie systematycznie nawadnianej roślinności domowej jest utrudnione przez wyjazdy, podróże służbowe oraz brak czasu w codziennym funkcjonowaniu. Problem dotyczy zarówno okazjonalnych hobbystów, jak i posiadaczy rozbudowanych, domowych kolekcji roślin (tzw. „urban jungle”). Nieprawidłowa pielęgnacja prowadzi do obumierania roślin, co przy rzadkich i kosztownych okazach oznacza wymierną stratę finansową, a dodatkowo zaburza estetykę i mikroklimat mieszkania. Dotychczasową praktyką bywa angażowanie osób trzecich (sąsiadów) do okresowego podlewania pod nieobecność domownika - rozwiązanie doraźne, kłopotliwe i zawodne. Zidentyfikowany problem wpisuje się w szerszy trend automatyzacji domowej (smart home) oraz rosnące zainteresowanie systemami IoT wspierającymi codzienną pielęgnację przestrzeni mieszkalnej bez konieczności posiadania przez użytkownika wiedzy technicznej.

B. Cel Projektu

Celem projektu było zaprojektowanie i wykonanie zintegrowanego systemu, który bez udziału człowieka monitoruje wilgotność gleby i automatycznie, precyzyjnie podlewa wskazane doniczki, raportując jednocześnie stan środowiskowy do chmury oraz umożliwiając zdalną kontrolę z dowolnego miejsca. W odróżnieniu od rozwiązań klasy „jedno urządzenie - jedna roślina”, projekt zakładał od początku model „jedna stacja - wiele roślin”, oparty na ruchomym ramieniu sterowanym w płaszczyźnie dwuwymiarowej (Pan-Tilt), zdolnym kierować strumień wody do dowolnie wybranej doniczki w swoim zasięgu. System docelowo miał być obsługiwany przez użytkownika o czysto konsumenckich

kompetencjach - bez konieczności konfiguracji sieciowej, logowania się do systemu czy posiadania wiedzy inżynierskiej.

II. STAN WIEDZY

A. Konstruktywna analiza istniejących rozwiązań

Zagadnienie automatyzacji nawadniania z użyciem Internetu Rzeczy jest przedmiotem intensywnych badań w skali rolniczej - Nsoh i in. (2024) przedstawiają systematyczny przegląd end-to-endowych, czasu rzeczywistego systemów nawadniania integrujących IoT i uczenie maszynowe, wskazując, że większość wdrożeń pozostaje wyspecjalizowana względem pojedynczego etapu przetwarzania danych (zbieranie, transmisja, decyzja), a pełna integracja end-to-end jest wciąż rzadkością. W skali mikrokontrolerowej, zbliżonej do niniejszego projektu, Hasib i Akib (2026) opisują system oparty na module ESP32 z czujnikami DHT22, HC-SR04 i wilgotności gleby, raportujący dane do chmury (ThingSpeak) - architektura sprzętowa jest bardzo zbliżona do WATIR IoT, jednak podobnie jak większość dostępnych na rynku rozwiązań konsumenckich obsługuje pojedynczą donicę. Wśród komercyjnych alternatyw dla gospodarstw domowych funkcjonują dwie kategorie: kule nawadniające (tanie, łatwo dostępne, lecz pozbawione precyzji, monitoringu i powiadomień, obciążone ryzykiem przelania lub przesuszenia) oraz dedykowane „smart doniczki”, oferujące precyzję kosztem bardzo wysokiej ceny jednostkowej w modelu 1:1, gdzie każde urządzenie obsługuje wyłącznie jedną roślinę.

B. Uzasadnienie i formy rozwiązania

Zidentyfikowana luka rynkowa dotyczy więc braku ekonomicznego rozwiązania łączącego precyzję i zdalny monitoring ze zdolnością obsługi wielu roślin z jednego urządzenia. Pytanie badawcze projektu można sformułować

następująco: czy możliwe jest zaprojektowanie jednej, niedrogiej stacji bazowej, która dzięki ruchomemu ramieniu 2D zapewni precyzyjne, zdalnie monitorowane podlewanie wielu niezależnych doniczek, przy jednoczesnym zachowaniu zerowego progu wejścia technicznego dla użytkownika końcowego? Zaproponowanym rozwiązaniem jest WATIR IoT: stacja bazowa z ramieniem Pan-Tilt, cyfrowym profilem opieki przypisanym do każdej rośliny (próg wilgotności, tryb pracy, docelowe kąty serw) oraz hybrydową architekturą sieciową eliminującą potrzebę logowania i konfiguracji sieciowej po stronie użytkownika..

III. OPIS ROZWIĄZANIA

A. Opis techniczny rozwiązania

System zrealizowano w architekturze trójwarstwowej (rys. 1). Warstwę sprzętową (Edge Control) stanowi jednostka Arduino UNO R4 WiFi, pełniąca funkcję centralnej jednostki decyzyjnej: obsługuje czujnik klimatu BME280 (magistrala SPI, CS = 10), ultradźwiękowy czujnik poziomu wody HC-SR04, analogowe sondy wilgotności gleby, sterownik silników L298N napędzający pompę wodną (PWM) oraz dwa serwomechanizmy ramienia Pan-Tilt. Warstwę chmurową stanowi konteneryzowany (Docker Compose) backend Node.js/Express.js wraz z relacyjną bazą PostgreSQL, odpowiedzialny za odbiór telemetry, przechowywanie historii oraz zarządzanie profilami roślin. Warstwę użytkową stanowi natywna aplikacja Android w architekturze MVVM (Kotlin, Jetpack Compose), komunikująca się przez bibliotekę Retrofit zarówno z API chmurowym, jak i bezpośrednio z lokalnym adresem IP urządzenia.

Kluczowym elementem architektury jest hybrydowy, dwusieczkowy model routingu. Ścieżka chmurowa obsługuje telemetry środowiskową (wysyłaną cyklicznie co 30 sekund) oraz konfigurację profili roślin - wprowadza umiarkowane opóźnienie, akceptowalne dla danych historycznych. Ścieżka lokalna obsługuje sterowanie wirtualnym joystickiem oraz żądania natychmiastowego podlewania: komendy trafiają bezpośrednio z aplikacji mobilnej na lokalny adres IP Arduino w sieci Wi-Fi, z pominięciem chmury, co gwarantuje znikome opóźnienie i pełną niezależność od łącza internetowego. Kontrakt danych między warstwami opiera się na obiektach JSON, np. `{“soil_moisture”: 40, “temp”: 24.5, “water_error”: false, “pan”: 90, “tilt”: 45}`. System zaprojektowano jako w pełni otwarty w sieci lokalnej/prywatnej. Świadomie zrezygnowano z modułów autentykacji (logowania, tokenów), aby wyeliminować barierę wdrożeniową dla przeciętnego użytkownika domowego. Wymagania funkcjonalne zdefiniowano na podstawie profilu użytkownika docelowego (osoba aktywna zawodowo, podróżująca, o czysto konsumenckich kompetencjach) i obejmują: (1) automatyczne podlewanie wyzwalane przekroczeniem progu wilgotności gleby zdefiniowanego per rośliną; (2) precyzyjne, niezależne adresowanie wielu doniczek jednym ramieniem Pan-Tilt; (3) zdalny podgląd stanu środowiskowego i historii pomiarowej; (4) ręczne sterowanie ramieniem z poziomu aplikacji. Do wymagań niezależnych od funkcji (niefunkcjonalnych) należą: niskie opóźnienie sterowania w czasie rzeczywistym (ścieżka lokalna), autonomia brzegowa (edge autonomy) - Arduino kontynuuje pracę na podstawie progów zapisanych w pamięci EEPROM

nawet po utracie łączności z chmurą lub aplikacją - skalowalność zapisu telemetry przy wysokiej częstotliwości próbkowania oraz zgodność z zasadami projektowania uniwersalnego (wysoki kontrast interfejsu, duże pola dotykowe, jednoznaczna sygnalizacja błędów), istotna wobec użytkowników starszych lub niedowidzących.

W toku realizacji napotkano i rozwiązano szereg problemów inżynierskich. Pierwotna architektura sprzętowa opierała się na układzie dwupłytkowym (Arduino Uno + moduł ESP8266 (ESP12-E z adapterem modułu, później moduł Wemos D1 Mini CH340) komunikujące się portem szeregowym 9600 baud) - w trakcie trwania kamienia milowego M3 zespół zmigrował system na jednopłytkowe rozwiązanie Arduino UNO R4 WiFi z wbudowanym koprocesorem komunikacyjnym, upraszczając architekturę i eliminując mostek szeregowy między płytkami.

Migracja ujawniła konflikt biblioteki WiFi3 z magistralą I2C - nawiązanie łączności Wi-Fi rekonfigurowało piny wykorzystywane przez czujnik BME280, uniemożliwiając stabilny odczyt klimatu; rozwiązaniem było przeniesienie czujnika na niezależną magistralę SPI oraz opóźnienie jego inicjalizacji do momentu nawiązania połączenia sieciowego. Niekontrolowane drgania ramienia Pan-Tilt wynikały z zakłóceń elektromagnetycznych generowanych przez antenę Wi-Fi oraz konfliktu przerwań systemowych między obsługą portu szeregowego a biblioteką Servo - rozwiązano to przez fizyczne odsunięcie modułu radiowego od serwomechanizmów oraz wdrożenie programowego uśredniania odczytów i strefy martwej (dead-zone).

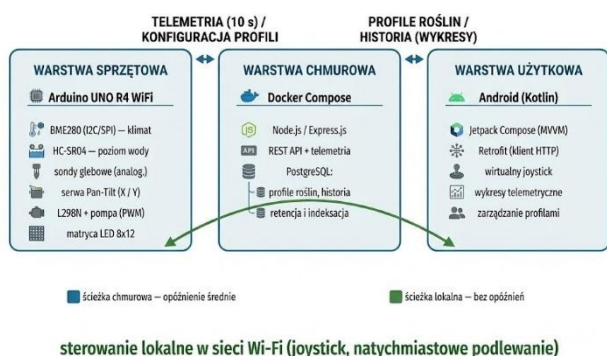
Równoczesny rozruch pompy wodnej i serwomechanizmów wywoływał chwilowe spadki napięcia zasilania, prowadzące zarówno do resetów mikrokontrolera, jak i fałszywych, zerowych odczytów z czujnika HC-SR04 (błędnie sygnalizujących brak wody). Warstwę zasilania rozdzielono na obwód logiczny (stabilne 5 V z przetwornicy Step-Down LM2596, skąd zasilane są sensory, wbudowane Arduino i serwomechanizmy) oraz obwód wykonawczy (pompa zasilana bezpośrednio z zasilacza 12 V) przy zachowaniu wspólnej masy, a w firmware wdrożono programowy filtr odrzucający nierealistyczne odczyty zerowe i podtrzymujący ostatnią prawidłową wartość poziomu wody.

Odrębnym wyzwaniem okazało się błędne wykrywanie adresu IP urządzenia przez serwer Node.js działający w kontenerze Docker - zamiast rzeczywistego adresu Arduino w sieci lokalnej odczytywana była brama sieciowa Dockera, co uniemożliwiało zwrotne sterowanie. Rozwiązaniem było wstrzykiwanie przez Arduino własnego adresu IP (odczytanego metodą `WiFi.localIP()`) bezpośrednio do przesyłanego ładunku JSON. Wreszcie, przy pracy równoległej pięciu osób w jednym repozytorium, wdrożono rygorystyczne reguły `.gitignore` oraz pliki szablonowe `.env.example`, zapobiegając wyciekowi danych konfiguracyjnych bazy danych oraz eliminując śledzenie ciężkich katalogów zależności.

B. Zarządzanie projektem i pracą zespołową

Pracę zespołu pięciu osób skoordynowano wyłącznie za pośrednictwem GitHub Projects, wykorzystując widok Roadmap (Gantt) do planowania osi czasu i zależności między kamieniami milowymi oraz widok tabelaryczny do przypisywania i śledzenia zadań. Harmonogram podzielono na

sześć kamieni milowych: M1 – ustalenie kontraktu danych i struktury bazy; M2 – kalibracja czujników i budowa konstrukcji Pan-Tilt; M3 – firmware i serwer lokalny (w trakcie którego przeprowadzono migrację na Arduino R4 WiFi); M4 – baza danych i konteneryzacja chmury; M5 – implementacja aplikacji mobilnej; M6 – integracja, testy end-to-end i walidacja fail-safe. Elastyczność narzędzia pozwoliła na płynną zmianę założeń architektonicznych bez zakłócenia harmonogramu prac pozostałych podzespołów. Cały proces deweloperski oparto na kontenerach Docker, ujednolicając środowisko uruchomieniowe niezależnie od systemu operacyjnego poszczególnych członków zespołu, a projekt aplikacji mobilnej uwzględnił zasady projektowania uniwersalnego (WCAG) już na etapie makiet w programie Figma.



Rysunek 1. Architektura systemu WATIR IoT: trzy warstwy (sprzętowa, chmurowa, użytkowa) oraz dwuścieżkowy model routingu - ścieżka chmurowa (telemetria/konfiguracja) i ścieżka lokalna (sterowanie w czasie rzeczywistym).

IV. BADANIA/TESTY/ŚRODOWISKO

Weryfikację systemu przeprowadzono w środowisku docelowym - fizycznym prototypie stacji bazowej z ramieniem Pan-Tilt, zbiornikiem wody i zestawem doniczek w jego zasięgu, połączonym z pełnym stosem chmurowym (Docker Compose) uruchomionym lokalnie oraz z aplikacją mobilną zainstalowaną na urządzeniu testowym w tej samej sieci Wi-Fi. Zakres testów funkcjonalnych obejmował poprawność logiki przypisywania profilu rośliny do parametrów sterujących (próg wilgotności, docelowe kąty serw Pan-Tilt) oraz kompleksowy test end-to-end typu fail-safe, w którym symulowano awarie (brak wody w zbiorniku, błąd odczytu czujnika) i weryfikowano poprawną reakcję całego łańcucha - sprzęt, backend, aplikacja. Test niefunkcyjny objął pomiar opóźnienia ścieżki lokalnej (czas odpowiedzi serwo mechanizmu na komendę joysticka) oraz 48-godzinny test ciągłości hydrologicznej, weryfikujący szczelność układu, stabilność zasilania i brak nieoczekiwanych resetów mikrokontrolera podczas długotrwałej, cyklicznej pracy.

V. WYNIKI I ANALIZA

Wyniki przeprowadzonych testów zestawiono w Tabeli I. Wszystkie cztery scenariusze zakończyły się wynikiem pozytywnym, co potwierdza poprawność przyjętych założeń architektonicznych. W szczególności test opóźnień potwierdził słusność hipotezy postawionej w rozdziale III.A: rozdzielenie ścieżki sterowania lokalnego od ścieżki telemetrycznej pozwala uzyskać reakcję ramienia na komendę joysticka odczuwalną jako

natychmiastową, niezależnie od obciążenia lub dostępności łącza internetowego, podczas gdy ścieżka chmurowa, obciążona większym, lecz akceptowalnym opóźnieniem, pozostaje w pełni wystarczająca dla danych telemetrycznych raportowanych z interwałem 30-sekundowym.

TABELA I
WYNIKI TESTÓW INTEGRACYJNYCH I FUNKCYJALNYCH SYSTEMU

Test	Zakres i metodyka	Wynik
Logika przypisywania profili roślin	Weryfikacja poprawności mapowania progu wilgotności i kątów Pan-Tilt zapisanych w profilu na parametry przesyłane do urządzenia	Zaliczony
End-to-End Fail-Safe	Symulacja awarii (brak wody, błąd czujnika) i ocena reakcji całego łańcucha: sprzęt - chmura - aplikacja	Zaliczony
Ciągłość hydrologiczna (48 h)	Nieprzerwana praca stacji z cyklicznym nawadnianiem przez 48 godzin (szczelność, stabilność zasilania)	Zaliczony
Opóźnienie ścieżki lokalnej (joystick)	Pomiar czasu odpowiedzi serwo mechanizmu na komendę z aplikacji mobilnej w sieci lokalnej	Potwierdzono znikome opóźnienie

Test end-to-end fail-safe wykazał, że system poprawnie wstrzymuje podlewanie i sygnalizuje awarię (m.in. za pośrednictwem diagnostycznej matrycy LED 8x12 oraz czerwonego banera błędu w aplikacji) w każdym z symulowanych scenariuszy, w tym przy fałszywych, zerowych odczytach czujnika HC-SR04 wywołanych spadkiem napięcia, potwierdzając skuteczność wdrożonego filtra programowego. Test 48-godzinny nie ujawnił nieszczelności ani nieoczekiwanych resetów układu, co pozwala ocenić rozwiązania zastosowane w warstwie zasilania (separacja logiki i napędów za pomocą przetwornicy LM2596) jako skuteczne. Ograniczeniem przyjętej metodyki weryfikacji pozostaje jej zakres, testy przeprowadził sam zespół projektowy na pojedynczym prototypie, bez zewnętrznego badania użyteczności z udziałem docelowych użytkowników; wynik potwierdza zatem wykonalność techniczną rozwiązania, nie stanowiąc jeszcze pełnej walidacji rynkowej.

VI. WNIOSKI I PERSPEKTYWY ROZWOJU

Projekt WATIR IoT zaowocował w pełni funkcjonalnym, zintegrowanym systemem IoT, który bezproblemowo łączy mikrokontroler, chmurę oraz natywną aplikację mobilną w spójną całość. Do mocnych stron rozwiązania należą: ekonomiczny model „jedna stacja – wiele roślin”, stanowiący realną alternatywę wobec kul nawadniających i kosztownych, dedykowanych smart doniczek; autonomia brzegowa, zapewniająca bezpieczną pracę systemu offline; hybrydowy, dwuścieżkowy routing, potwierdzony testami jako skutecznie łączący niskie opóźnienie sterowania z asynchronicznym zapisem historii; a także zdolność zespołu do adaptacji architektury sprzętowej (migracja dwupłytkowa → jednopłytkowa) w odpowiedzi na rzeczywiste ograniczenia techniczne, bez zakłócenia harmonogramu.

Do słabszych stron, świadomie przyjętych jako uzasadnione uproszczenia, należą: całkowity brak modułu autentykacji, co ogranicza zastosowanie do sieci prywatnej/domowej; obsługa kolejnych doniczek przez pojedyncze ramię w sposób

sekwencyjny, a nie równoległy; oraz zależność natychmiastowego sterowania od dostępności lokalnej sieci Wi-Fi.

Weryfikacja funkcjonalna i wnioski z rozdziału V wskazują trzy naturalne kierunki dalszego rozwoju: integrację z ekosystemem smart home poprzez protokół MQTT i Home Assistant; wprowadzenie uczenia maszynowego do inteligentnego doboru dawki wody na podstawie historycznych trendów wilgotności i temperatury z czujnika BME280; oraz skalowanie sprzętowe stacji o dedykowany układ elektrozaworów, umożliwiający równoległą obsługę większej liczby roślin na większych dystansach.

Realizacja projektu zgodnie z metodologią Design Thinking, od badania potrzeb rynkowych, przez projekt mechaniki i elektroniki, po backend chmurowy i mobilny UX, dała zespołowi praktyczne doświadczenie pełnego cyklu wytwórczego produktu IoT.

ZAŁĄCZNIKI

Repozytorium kodu źródłowego projektu (firmware, backend, aplikacja mobilna) - https://github.com/BaDaQu/WATIR_IoT

Materiał wideo z demonstracją działania stacji - <https://tinyurl.com/WATIRIOT>

PODZIĘKOWANIA

Zespół projektowy dziękuje Panu dr. inż. Jackowi Nazdrowiczowi za merytoryczną opiekę nad projektem, cenne wskazówki inżynierskie na etapie integracji warstwy sprzętowej z chmurą oraz wsparcie w rozwiązywaniu problemów napotkanych w trakcie realizacji.

BIBLIOGRAFIA

Hasib, A., Akib, A. S. M. A. S. (2026). An IoT-based smart plant monitoring and irrigation system with real-time environmental sensing, automated alerts, and cloud analytics. arXiv, arXiv:2601.15830.

Nsoh, B., Katimbo, A., Guo, H., Heeren, D. M., Nakabuye, H. N., Qiao, X., Ge, Y., Rudnick, D. R., Wanyama, J., Bwambale, E., Kiraga, S. (2024). Internet of Things-based automated solutions utilizing machine learning for smart and real-time irrigation management: A review. Sensors, 24(23), 7480, doi: 10.3390/s24237480

Skeen J., Greenhalgh D. (2019). Programowanie w języku Kotlin. The Big Nerd Ranch Guide, HELION, Gliwice, s. 408

WKŁAD W PRACĘ NAD PROJEKTEM



Wojciech Stochmiałek, student 6. semestru kierunku Informatyka na Wydziale Elektroniki, Elektrotechniki, Informatyki i Automatyki (WEEIA) Politechniki Łódzkiej, odpowiedzialny za warstwę sprzętową projektu. Zaprojektował obwód zasilania z separacją masy logicznej i wykonawczej (z wykorzystaniem przetwornicy step-down LM2596), złożył ruchome ramię Pan-Tilt wraz z fizycznym joystickiem oraz przeprowadził montaż i kalibrację czujników gleby, BME280 i HC-SR04. Opracował algorytm kinematyki przeliczający pozycję joysticka

na kąty serwomechanizmów oraz mechanizm zabezpieczenia pompy przed pracą na sucho. Odpowiadał za zapis pozycji ramienia w pamięci nieulotnej (EEPROM) oraz końcowy montaż i izolację obudowy przed wilgocią. Współtworzył testy integracyjne, w tym 48-godzinny test ciągłości hydrologicznej.



Bartłomiej Raj, student 6. semestru kierunku Informatyka na Wydziale Elektroniki, Elektrotechniki, Informatyki i Automatyki (WEEIA) Politechniki Łódzkiej, pełniący funkcję lidera zespołu oraz odpowiedzialny za warstwę sieciową systemu. Skonfigurował repozytorium projektu i tablicę Kanban w GitHub Projects, koordynując pracę pięcioosobowego zespołu w metodyce zwinnej. Współtworzył kontrakt danych JSON wymieniających między warstwami, skonfigurował moduł sieciowy jako serwer oraz zintegrował komponenty chmurowe z warstwą firmware, w tym klienta API w aplikacji mobilnej i logikę sterowania z joysticka. Nadzorował migrację architektury sprzętowej na Arduino UNO R4 WiFi w trakcie trwania kamienia milowego M3 oraz współprzewodził testy integracyjne i 48-godzinny test ciągłości hydrologicznej.



Marcel Podlecki, student 6. semestru kierunku Informatyka na Wydziale Elektroniki, Elektrotechniki, Informatyki i Automatyki (WEEIA) Politechniki Łódzkiej, odpowiedzialny za warstwę UI/UX aplikacji mobilnej. Zaprojektował makiety interfejsu w programie Figma oraz spójny motyw wizualny aplikacji, zgodny z zasadami projektowania uniwersalnego (wysoki kontrast, duże pola dotykowe, jednoznaczna sygnalizacja błędów). Zaimplementował warstwę prezentacji w Jetpack Compose, obejmującą ekran startowy i nawigację, panel główny (dashboard), ustawienia oraz interfejs wirtualnego joysticka. Współtworzył moduł wykresów telemetrycznych oraz system powiadomień o błędach, a także uczestniczył w końcowych testach integracyjnych typu E2E i poprawkach interfejsu przed oddaniem projektu.



Bartłomiej Jedyk, student 6. semestru kierunku Informatyka na Wydziale Elektroniki, Elektrotechniki, Informatyki i Automatyki (WEEIA) Politechniki Łódzkiej, odpowiedzialny za warstwę backendową i bazodanową systemu. Zaprojektował strukturę relacyjnej bazy danych PostgreSQL, w tym tabele telemetrii i profili roślin wraz z indeksami optymalizującymi zapytania chronologiczne. Zaimplementował REST-owe API serwera Node.js/Express, obsługujące telemetrię, sterowanie oraz pełny CRUD profili roślin, po czym skonteneryzował środowisko za pomocą Docker Compose, ujednoliciając konfigurację uruchomieniową w zespole. Wdrożył asynchroniczny algorytm retencji danych, zabezpieczający bazę przed przepełnieniem przy wysokiej częstotliwości próbkowania. Współpracował przy konfiguracji komunikacji sieciowej między firmware a chmurą oraz przy testach logiki przypisywania profili.



Dominik Pokorski, student 6. semestru kierunku Informatyka na Wydziale Elektroniki, Elektrotechniki, Informatyki i Automatyki (WEEIA) Politechniki Łódzkiej, odpowiedzialny za logikę aplikacji mobilnej. Zaimplementował warstwę komunikacyjną w architekturze MVVM z użyciem biblioteki Retrofit, obsługującą zarówno API chmurowe, jak i bezpośrednią komunikację z lokalnym adresem IP urządzenia. Opracował obsługę asynchronicznych wątków wirtualnego joysticka oraz logikę sterowania w czasie rzeczywistym, a także wdrożył wykresy telemetryczne prezentujące historyczne dane środowiskowe. Współtworzył ekran zarządzania profilami roślin (CRUD) oraz system powiadomień o błędach. Uczestniczył w testach logiki przypisywania profili oraz w kompleksowym teście integracyjnym typu End-to-End.



dr inż. Jacek Nazdrowicz, Politechnika Łódzka, Katedra Mikroelektroniki i Techniki Informatycznych, Żeromskiego 116, 90-924 Łódź, E-mail: jacek.nazdrowicz@p.lodz.pl, członek IEEE, Pracownik naukowy Politechniki Łódzkiej specjalizujący się w tematyce układów scalonych (Electronics Integrated Circuits). Swoją działalność akademicką i badawczą z sukcesem łączy z wieloletnim doświadczeniem w globalnym przemyśle IT, pracując na stanowisku Senior Domain Architect w firmie Fujitsu, gdzie uzyskał prestiżowy tytuł Global Fujitsu Distinguished Engineer. Specjalizuje się w architekturze serwerowej,

sieciowej, systemach serwerowych Microsoft (Windows i SQL Server) oraz technologiach IBM Tivoli. Posiada bogate, praktyczne umiejętności w zakresie programowania w językach C++, Python, Java oraz Visual Basic. W swojej pracy dydaktycznej oraz inżynierskiej łączy zaawansowaną wiedzę teoretyczną i trendy badawcze z praktycznym, rynkowym podejściem do projektowania, wdrażania oraz optymalizacji złożonych systemów informatycznych.